The Complete Beginner's Guide

Epsilon IDE Engine — Potato PC Edition

Understanding Every Single Part From Zero

No AI/ML Background Needed	No C++ Knowledge Needed	No Low-Level Experience Needed	Complete Code Included

This guide explains everything — from what RAM actually is, to how the AI model sits inside your GPU, to how the programs talk to each other, to how you write each piece of code. If you can write basic Python, you can build this.

2 GB RAM + 2 GB VRAM

Table of Contents

PART 1	Starting From Zero — What Is This Project?
PART 2	Memory 101 — RAM, VRAM, and SSD Explained Simply
PART 3	What Is an AI Model? Weights, Tokens, and Inference
PART 4	How the Model Gets Into VRAM — The Loading Process
PART 5	BitNet — How We Compress a Model by 8×
PART 6	The KV Cache — The Model's Short-Term Memory
PART 7	How All the Programs Talk to Each Other
PART 8	The Complete Request Flow — From Your Keyboard to the Answer
PART 9	PicoClaw Agents — Four Specialists Doing One Job Each
PART 10	Clara Context Oracle — Searching Code Without a Neural Model
PART 11	The C++ Extensions — What They Do (Without Writing C++)
PART 12	tinygrad and Sparse Attention — Shrinking the Memory Footprint
PART 13	Python Concepts You Need — A Practical Guide
PART 14	Setting Up Your Machine — Complete Installation
PART 15	Building Component 1: The BitNet Model Wrapper
PART 16	Building Component 2: The PicoClaw Orchestrator
PART 17	Building Component 3: Clara Context Oracle
PART 18	Building Component 4: Aether Link
PART 19	Building Component 5: The Main Engine
PART 20	Testing, Debugging, and Monitoring
GLOSSARY	Every Technical Term Explained in Plain English

PART 1 Starting From Zero What Is This Project, Exactly?

1.1 The One-Sentence Summary

We are building an AI coding assistant — similar to GitHub Copilot — that runs entirely on a computer with only 2 gigabytes of RAM and 2 gigabytes of GPU memory. No internet connection needed. No subscription. No cloud. It runs on your hardware, locally, using a tiny but surprisingly capable AI model.

1.2 Why Is This Hard?

A normal AI coding assistant like Copilot runs on massive server farms with hundreds of gigabytes of memory. The AI model alone — the thing that generates the code suggestions — typically requires 6 to 14 gigabytes just to sit in memory. We have 2 gigabytes total. That is not a small gap. That is a fivefold shortfall, and closing it requires rethinking every single component of the system.

■ Think of it like this: Copilot uses a stretch limousine to carry its AI. We need to fit the same passenger into a motorbike and actually go faster because our engine is more efficient. It sounds impossible — and that is exactly what makes it interesting.

1.3 The Five Technologies That Make It Possible

Each technology in this stack solves one specific problem. Together they turn the impossible into something that fits comfortably in your laptop.

BitNet 1.58-bit weights	Normally, an AI model stores each of its 494 million parameters as a 16-bit number (like 0.3742). BitNet stores each one as just –1, 0, or +1. This shrinks the model from 988 MB down to 120 MB. The model was trained from scratch to use these tiny numbers, so quality barely suffers.
bitnet.cpp (the inference engine)	Instead of using PyTorch (which consumes 800 MB of RAM before doing anything), we use a lean C++ program called bitnet.cpp. It runs as a separate process, does all the AI computation, and talks to our Python code through a simple HTTP call. The whole executable is about 30 MB.
tinygrad + Sparse Attention	When the model is generating text, it keeps a 'memory' of everything it has seen so far — called the KV cache. Standard attention makes this cache grow with the square of the context length. Sparse attention limits each token to looking at only the 64 most relevant previous tokens, cutting memory from 440 MB to 80 MB.
PicoClaw Micro-Agents	Instead of one large model doing everything, four small specialised 'agents' divide the work. The Router uses simple keyword matching (no AI). The Recall agent uses classical search (no AI). Only the Coder agent actually calls the model. This means three out of four tasks are solved with zero AI inference cost.

sqlite-vec + TF-IDF	Finding relevant code in your project normally requires a neural embedding model (another 500 MB). We use TF-IDF — a classical algorithm from the 1970s — to convert code into searchable vectors. The index lives on disk. Only the active search result ever touches RAM. Cost: about 20 MB.

1.4 What You Will Be Able to Do When Finished

Once you have built this project, VS Code will show AI-generated code suggestions as you type — just like Copilot, but running entirely on your own hardware. You type a function signature and the system completes the body. You describe what you want in a comment and it writes the code. It searches your existing codebase for relevant context and includes it automatically. All of this happens in 1 to 3 seconds on a laptop CPU.

PART 2 Memory 101 RAM, VRAM, and SSD Explained Simply

2.1 The Filing Cabinet, the Desk, and the Workbench

Your computer has three places to store things, and understanding the difference between them is the single most important concept for this entire project. Everything else — why we use bitnet.cpp, why we keep the model in VRAM, why swap space matters — follows from understanding this.

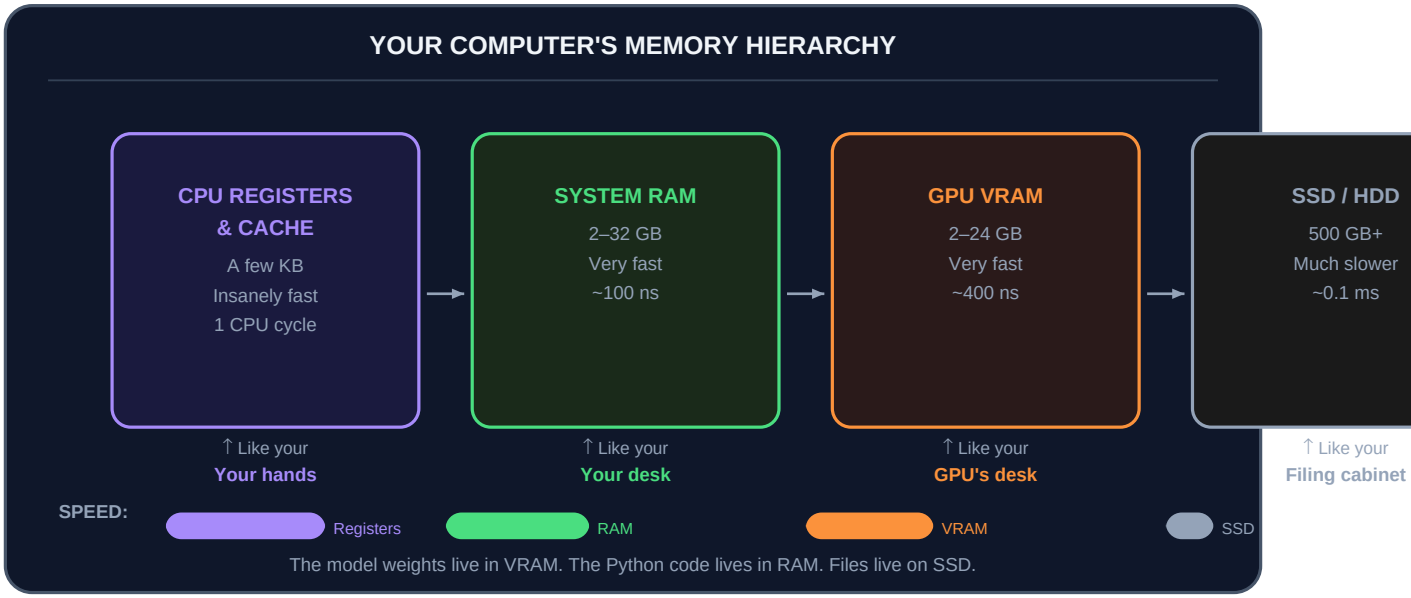

Figure 1: The four levels of memory in your computer, from fastest to slowest.

2.2 RAM — Your Desk

System RAM (Random Access Memory) is where your computer keeps everything it is currently working with. When you open a program, it loads from the SSD into RAM. When you close it, it leaves RAM. RAM is fast — reading from it takes about 60–100 nanoseconds. It is also temporary: when you turn off the computer, everything in RAM is gone. We have 2 GB of this. The entire Python runtime, the OS, the PicoClaw agents, and all our working data must fit in that 2 GB.

> ■ ***Analogy:*** *RAM is your desk. You can only work on things that are on the desk. A bigger desk means you can work on more things simultaneously. Our desk is small — we keep it ruthlessly tidy.*

2.3 VRAM — The GPU's Dedicated Workbench

VRAM (Video RAM) is memory that belongs exclusively to your GPU (graphics card). The GPU cannot easily use regular RAM — every access would have to travel over the PCIe bus, which is 30× slower than the GPU's internal VRAM bus. By keeping the AI model in VRAM, the GPU can process it at full speed. We have 2 GB of VRAM. The model weights (120 MB) live here permanently. The KV cache (80 MB) lives here too. The rest is empty headroom.

■ ***Analogy:*** *VRAM is a specialised workbench next to your desk, dedicated to GPU work. The GPU can only pick things up from its own workbench — not from your main desk. Putting the model on the GPU's workbench is the whole reason generation is fast.*

2.4 SSD — The Filing Cabinet

The SSD (Solid State Drive) is where files live permanently. It is much larger than RAM or VRAM, but also much slower. Reading from an SSD takes 0.1 milliseconds. Reading from RAM takes 0.0001 milliseconds. That is a thousand times slower. In this project, the SSD stores: the model file (loaded once at startup), the sqlite-vec code index (searched on demand), and the 4 GB swap file (used only in emergencies). The model is loaded from SSD into RAM and VRAM once when the engine starts, then never again.

2.5 Why Memory Limits Are Different from Storage Limits

This is the confusion that trips people up most often. Your laptop might have 256 GB of SSD storage, but only 2 GB of RAM. Running programs do not use SSD storage — they use RAM. The model file is 120 MB on disk, but it must be loaded into RAM and then copied into VRAM to be used. If there is no space in VRAM, the model cannot run — it does not matter how much free SSD space you have.

✓ Concrete numbers: Our 2 GB VRAM holds the 120 MB model + 80 MB KV cache + 50 MB activations + 100 MB CUDA overhead = 350 MB used out of 2048 MB available. We have 1.7 GB of VRAM just sitting empty. That is our safety margin.

2.6 What Happens When Memory Runs Out

When RAM fills up, the OS moves the coldest pages to the swap file on SSD. This is called 'swapping' and it is 1,000× slower than RAM. The program keeps running but becomes very slow. When VRAM fills up, you get a CUDA Out Of Memory (OOM) error. This is more serious — it can crash the CUDA driver, requiring a reboot. This is why we are so precise about memory budgeting. Every megabyte is deliberate.

PART 3 What Is an AI Model? Weights, Tokens, and Inference

3.1 The Simplest Possible Explanation

An AI language model is, at its core, a very large mathematical function. You give it a sequence of numbers as input. It does billions of simple arithmetic operations on those numbers. It gives you a new number as output. That output number represents the most likely next word. You add that word to the input and repeat. That is the entire process. The art is in the 494 million 'weights' — the numbers that define how the arithmetic transforms your input into an output.

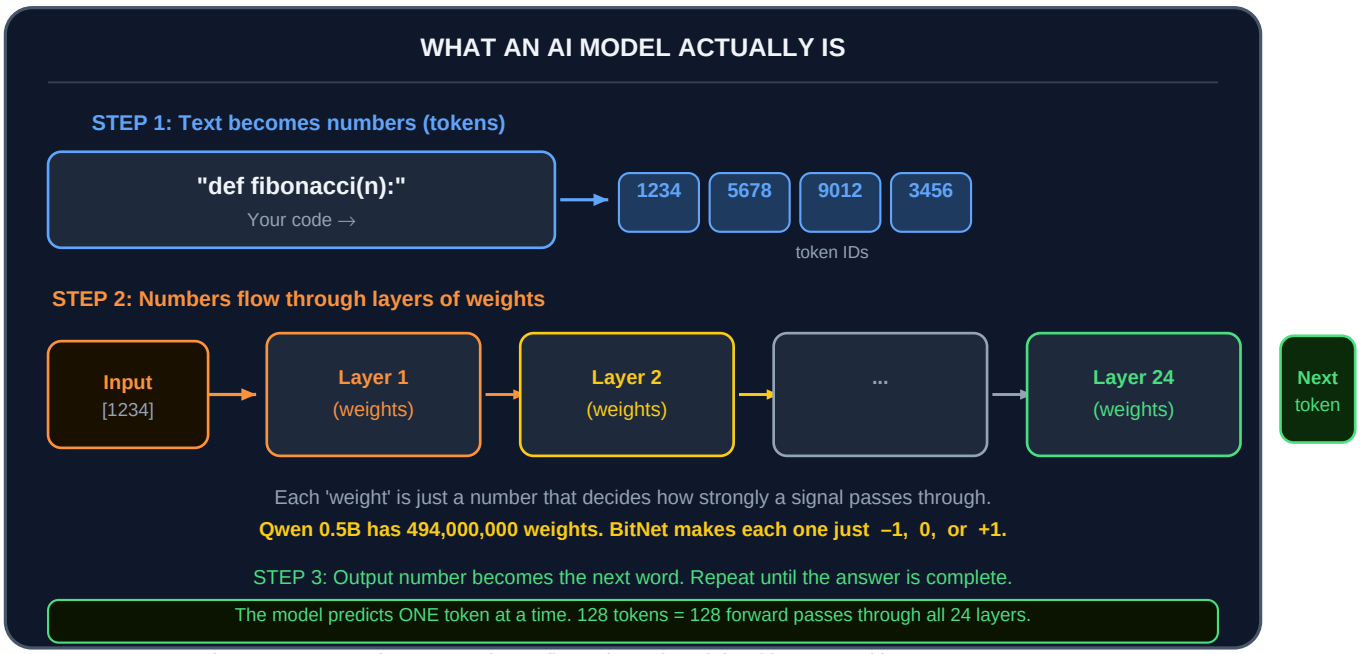

Figure 2: How text becomes tokens, flows through weighted layers, and becomes output.

3.2 What Are Tokens?

Models do not understand letters or words directly. They understand 'tokens' — chunks of text that have been assigned a number. The word 'fibonacci' might be token number 37,429. The characters 'def' might be token number 1,234. Qwen 0.5B knows 151,936 different tokens — enough to cover English, Chinese, most programming languages, and common punctuation. Before processing your text, it is converted from characters to a list of token numbers (tokenisation). After generating output, the token numbers are converted back to text (detokenisation).

> ■ One token is roughly 0.75 words. Our context limit of 512 tokens is approximately 380 words — enough for a function signature, a few lines of surrounding code, and a code suggestion. We keep it short deliberately to save KV cache memory.

3.3 What Are Weights?

Weights are the 494 million numbers that define the model's behaviour. They were determined during training — a process where the model processed trillions of tokens of code and text, and each weight was nudged

slightly to make the model's predictions more accurate. Once training is complete, the weights are frozen. They do not change while the model is being used (inference). The entire model file is just these 494 million numbers stored compactly on disk.

> ■ ***Analogy:*** *Weights are like the settings on a mixing board. Each one controls how much a signal passes through a particular channel. Training found the settings that make the output sound like high-quality code. Inference just plays the music.*

3.4 What Is Inference?

Inference is using the model to generate output — as opposed to training, which adjusts the weights. During inference: (1) your input is tokenised, (2) the tokens flow through all 24 transformer layers, (3) the final layer outputs probabilities for every token in the vocabulary, (4) the most likely next token is selected, (5) it is added to the input, and (6) the whole process repeats until the output is complete. Each repetition of steps 1-6 generates exactly one token. Generating 128 tokens means 128 complete passes through all 24 layers.

3.5 What Are Transformer Layers?

A transformer layer is one stage of processing. Qwen 0.5B has 24 layers. Each layer has two parts: self-attention (which lets the model look at other tokens for context) and a feed-forward network (which processes each token independently). The weights in each layer are what was learned during training. You do not need to understand the mathematics — just know that each layer refines the representation of the input, and by layer 24 the model has a good prediction for what comes next.

PART 4 How the Model Gets Into VRAM The Loading Process, Step by Step

Understanding exactly how the model moves from a file on your SSD into the GPU's memory is crucial. It demystifies what bitnet.cpp does, why startup takes a couple of seconds, and why generation is fast after that.

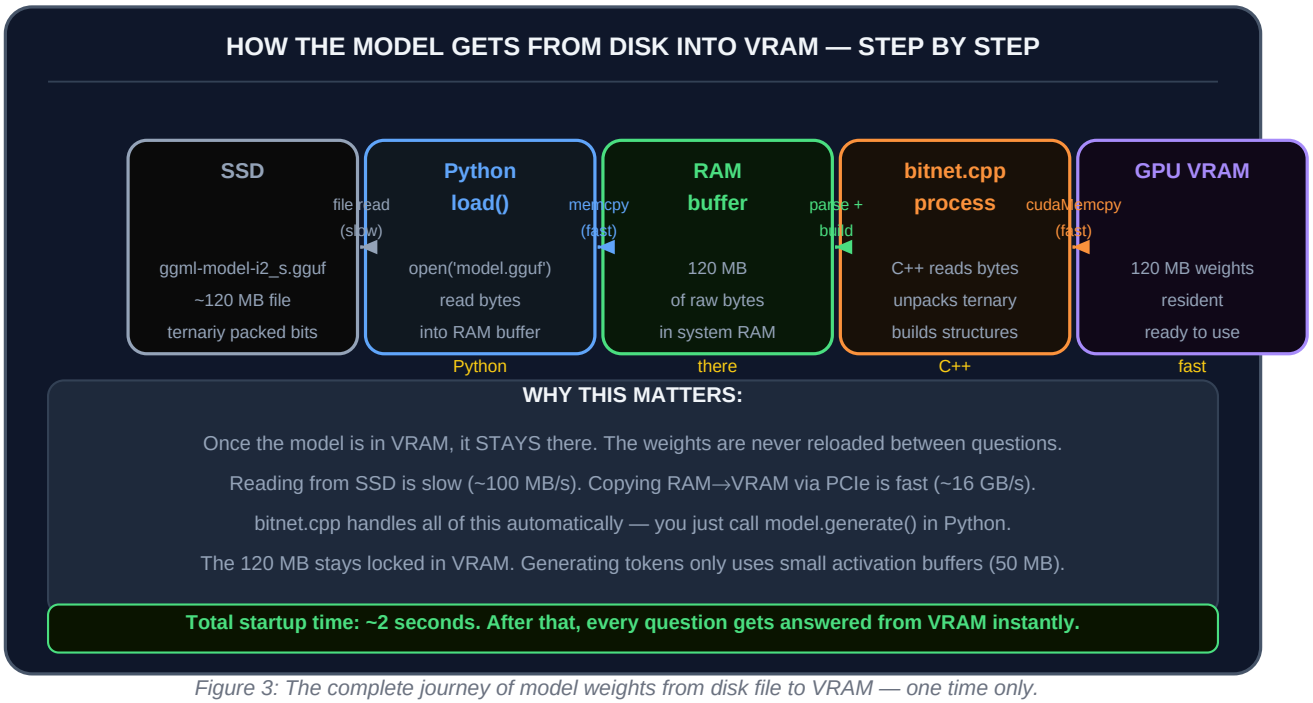

Figure 3: The complete journey of model weights from disk file to VRAM — one time only.

4.1 The GGUF File Format

The model is stored in a file format called GGUF (GPT-Generated Unified Format). This is essentially a compact binary file containing: a header with model configuration (number of layers, attention heads, vocabulary size, etc.) and then the raw weight bytes packed as efficiently as possible. For our BitNet model, the weights are packed four-to-a-byte in 2-bit format (since each weight is only –1, 0, or +1). The entire file is about 120 MB — small enough to load in under 2 seconds.

4.2 What bitnet.cpp Does at Startup

When you start the bitnet.cpp server, it does the following automatically: reads the GGUF file from disk, parses the header to understand the model architecture, unpacks the ternary weight bytes into its internal data structures, and if GPU layers are enabled, copies the weights to VRAM via cudaMemcpy. In CPU mode (which we use on potato hardware), the weights stay in RAM and the GPU is not involved at all. The SIMD-optimised CPU kernels are fast enough.

■ In our setup, GPU layers are set to 0. This means everything runs on CPU. The VRAM is used only for the tinygrad KV cache (80 MB). The 120 MB model weights live in RAM. This is intentional — bitnet.cpp's CPU kernels are actually faster than naive GPU execution for a 0.5B model.

4.3 Why the Model Stays Loaded

Unlike a web server that shuts down between requests, the bitnet.cpp server process stays running continuously. The model weights are loaded once and kept in memory for the entire session. When you ask for a code suggestion, the weights are already there — the request just needs to flow through them. This is why the second request is as fast as the first. Loading the model is the slow part (about 2 seconds). Each inference is fast (about 1–3 seconds for 128 tokens).

4.4 What Happens During Each Token Generation

When bitnet.cpp generates a token, the process is: (1) the input tokens are looked up in the embedding table (40 MB, also in memory), (2) the resulting vectors pass through all 24 layers — in each layer, the weights are multiplied against the input (or rather, added/subtracted since they are ternary), (3) the output of the final layer is passed through a softmax function to get probabilities over all 151,936 tokens, (4) the highest-probability token is selected (or sampled if temperature > 0), (5) that token is appended to the KV cache and output as text. Steps 1–5 take about 25 milliseconds per token on a 4-core CPU.

PART 5 BitNet 1.58-bit How We Compress a Model by 8×

5.1 The Problem With Normal Model Weights

A standard neural network weight is stored as a 16-bit floating-point number. 16 bits means the number can take one of 65,536 different values — enough precision to store something like 0.37428192. Storing 494 million of these numbers costs 494,000,000 × 2 bytes = 988 MB. That does not fit in our 2 GB budget alongside everything else. We need to make the model much smaller.

5.2 BitNet's Radical Insight

BitNet, developed by Microsoft Research, asks a provocative question: what if we only allowed weights to be one of three values: –1, 0, or +1? This sounds like it would destroy the model's intelligence. But at 500 million parameters and above, something surprising happens: the model learns to express complex transformations using only these three values, by combining them in sophisticated patterns across millions of weights. The key insight is that BitNet models are trained from scratch with this constraint — the model learns to be accurate within these limits rather than being forced into them after the fact (which would destroy quality).

5.3 The Storage Math

Three possible values (–1, 0, +1) need log■(3) = 1.58 bits to represent. In practice, we use 2 bits per weight and pack four weights into one byte. The encoding is simple: 00 = –1, 01 = 0, 10 = +1. So 494,000,000 weights need 494,000,000 × 0.25 bytes = 123.5 MB. Plus a small overhead for embedding tables (40 MB) which stay in FP16 (the first and last layers lose too much quality if quantised). Total: about 163 MB — compared to 988 MB for the FP16 version. That is a 6× reduction in model size.

5.4 Why Inference Is Faster Too

The memory saving is not the only benefit. BitNet is also faster to run. Standard neural network inference requires multiply-accumulate operations: for each weight, multiply it by the input and add the result to an accumulator. Multiplication is expensive. BitNet eliminates it completely. If the weight is +1: just add the input. If the weight is –1: just subtract the input. If the weight is 0: do nothing at all. No multiplication. Just addition, subtraction, or skip. These operations are 4× cheaper than floating-point multiplication, and bitnet.cpp's SIMD kernels process 64 weights per CPU instruction.

> ■ ***Analogy:*** *Imagine you need to mix a cocktail, and your recipe has 494 million ingredients with precise measurements like '37.4 ml'. BitNet says: each ingredient is either add a full shot, add nothing, or subtract a full shot. Simpler instructions, but so many of them that the drink tastes the same — possibly better because the simpler rules are easier to optimise.*

5.5 Quality Loss — Is It Noticeable?

On research benchmarks, BitNet 0.5B matches or slightly trails the FP16 version on language understanding tasks. For code completion, the difference is minimal. The model can still complete functions, understand

variable names, follow docstrings, and produce syntactically valid code. It will occasionally produce a subtly wrong algorithm for complex logic — but this is true of all 0.5B models regardless of quantisation. The CRITIC agent's Tree-sitter check catches syntax errors and triggers re-generation.

PART 6 The KV Cache The Model's Short-Term Memory

Every time the model generates a token, it needs to consider all the tokens that came before it. Without optimisation, this would require re-processing the entire input from scratch for every single token — getting exponentially slower as the output grows. The KV cache solves this completely.

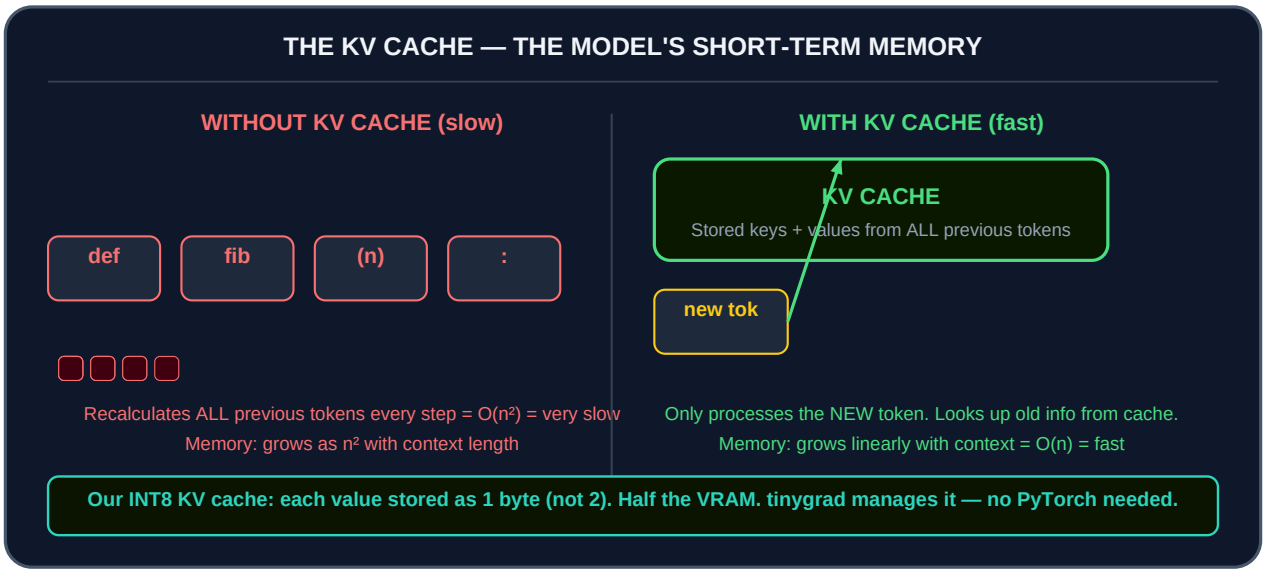

Figure 4: Full O(n²) attention versus sparse O(n·k) attention with INT8 KV cache.

6.1 Keys and Values — What Are They?

In each transformer layer, the self-attention mechanism computes three things for each token: a Query (what this token is looking for), a Key (what this token offers to others), and a Value (the actual content this token contributes). When generating token N+1, the model needs the Keys and Values from all previous N tokens to compute attention. The KV cache stores these K and V vectors so they do not need to be recomputed. Only the Query for the new token needs fresh computation.

6.2 The Memory Cost

For Qwen 0.5B with a 512-token context: 2 (K and V) × 24 (layers) × 14 (attention heads) × 512 (tokens) × 64 (dimensions) × 2 (bytes FP16) = approximately 440 MB. That is enormous for our budget. We attack this in two ways: switch to INT8 (halving to 220 MB) and use sparse attention (attending to only 64 of 512 tokens, halving again to 110 MB). Combined: the KV cache costs about 80 MB — acceptable.

6.3 Sparse Attention — Attending to Fewer Tokens

Standard attention has every token look at every other token — O(n²) cost. Sparse attention has each token look at only the top-K most relevant other tokens. We use K=64, meaning each new token only attends to the 64 previous tokens it finds most similar (based on query-key dot product). For code completion, this works very well — relevant context is usually nearby (the current function) or a few well-known symbols (imports at the top). Attending to all 512 tokens adds almost nothing for typical code.

6.4 Why tinygrad Instead of PyTorch?

PyTorch is the standard framework for machine learning, but it is extremely heavy — it imports 800 MB to 1.2 GB of Python packages and CUDA libraries just to start up. We do not need all that machinery. We only need tensor operations for the KV cache. tinygrad is a much smaller framework that handles exactly this. It imports in under 0.3 seconds and uses about 50 MB of RAM idle. The INT8KVCache class is about 30 lines of tinygrad code and replaces an entire PyTorch dependency.

PART 7 How the Programs Talk to Each Other IPC, Pipes, and Queues

This project is not one program — it is several programs running simultaneously, passing messages to each other. Understanding how they communicate is key to understanding the architecture and debugging problems when they occur.

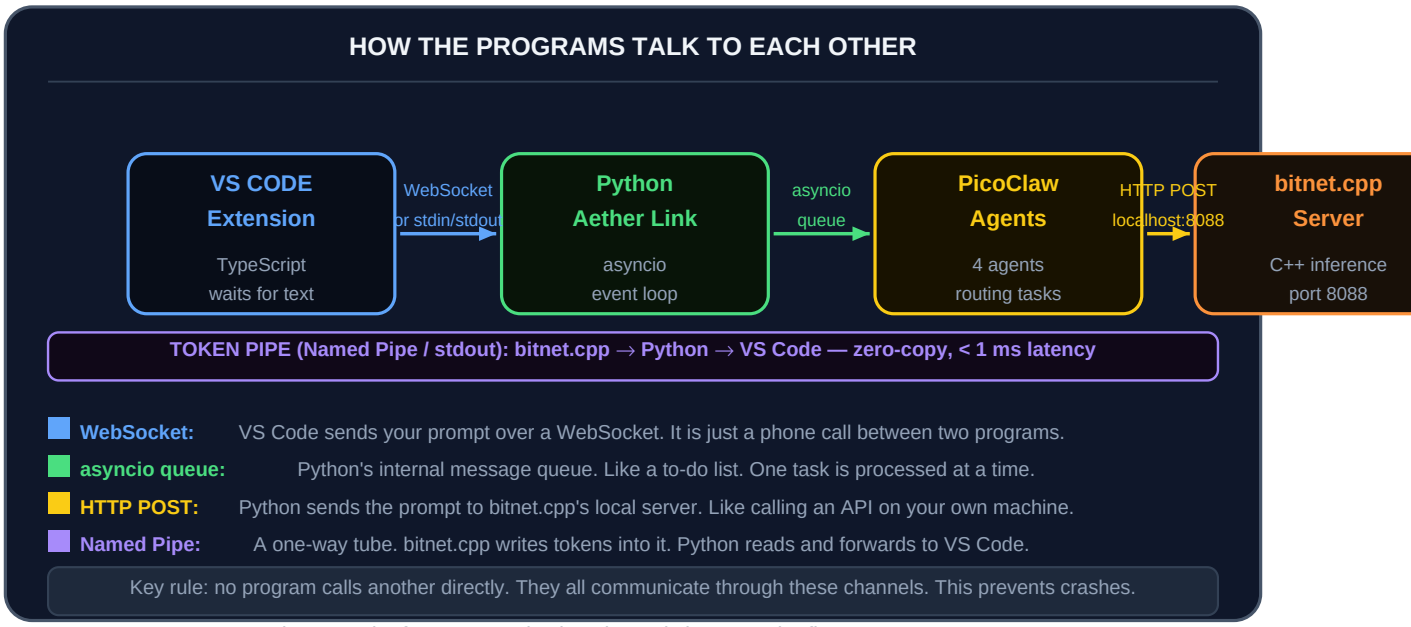

Figure 5: The four communication channels between the five components.

7.1 Why Separate Programs?

The most important reason: bitnet.cpp is written in C++, not Python. Python cannot import C++ code directly without a wrapper (pybind11). Running bitnet.cpp as a separate server process avoids this complexity — Python just makes HTTP calls to it like calling an API. The second reason: if bitnet.cpp crashes (CUDA OOM, segfault), it does not take the Python process with it. The engine can detect the crash and restart it automatically.

7.2 stdin and stdout — The Simplest Communication

Every program has a standard input (stdin) and standard output (stdout). stdin is like a keyboard — programs read text from it. stdout is like a screen — programs write text to it. We use these to communicate between VS Code and the Python engine: VS Code writes a JSON request to the engine's stdin, and the engine writes a JSON response to stdout. This eliminates the need for a web server entirely — no Flask, no FastAPI, no HTTP overhead.

■ Example stdin → stdout exchange: VS Code writes: {"prompt": "def fibonacci(n):"} Python engine reads this, processes it, and writes back: {"ok": true, "result": " if n <= 1: return n\n return fibonacci(n-1) + fibonacci(n-2)"}

7.3 HTTP — Talking to bitnet.cpp

bitnet.cpp runs as a local HTTP server on port 8088. HTTP is the same protocol used to load web pages — it is a request-response protocol. Python sends a POST request (like submitting a form) with the prompt and parameters. bitnet.cpp responds with the generated text. Because this is local (on the same machine), the latency is under 1 millisecond — the HTTP overhead is negligible compared to the inference time.

7.4 The asyncio Event Loop — Non-Blocking Waiting

Python normally runs one thing at a time. If it is waiting for bitnet.cpp to finish generating (which takes 1–3 seconds), it cannot do anything else — it is 'blocked'. asyncio solves this with an event loop: instead of blocking, Python registers a 'when this is done, call this function' callback and goes back to waiting for other events. This lets the engine handle new requests while waiting for the current generation to finish. The 'async def' and 'await' keywords in our code are the markers for this.

PART 8 The Complete Request Flow From Your Keyboard to the Answer

This is the single most important thing to understand about the project. If you can trace a request from start to finish, every piece of the code makes sense because you know what it is for.

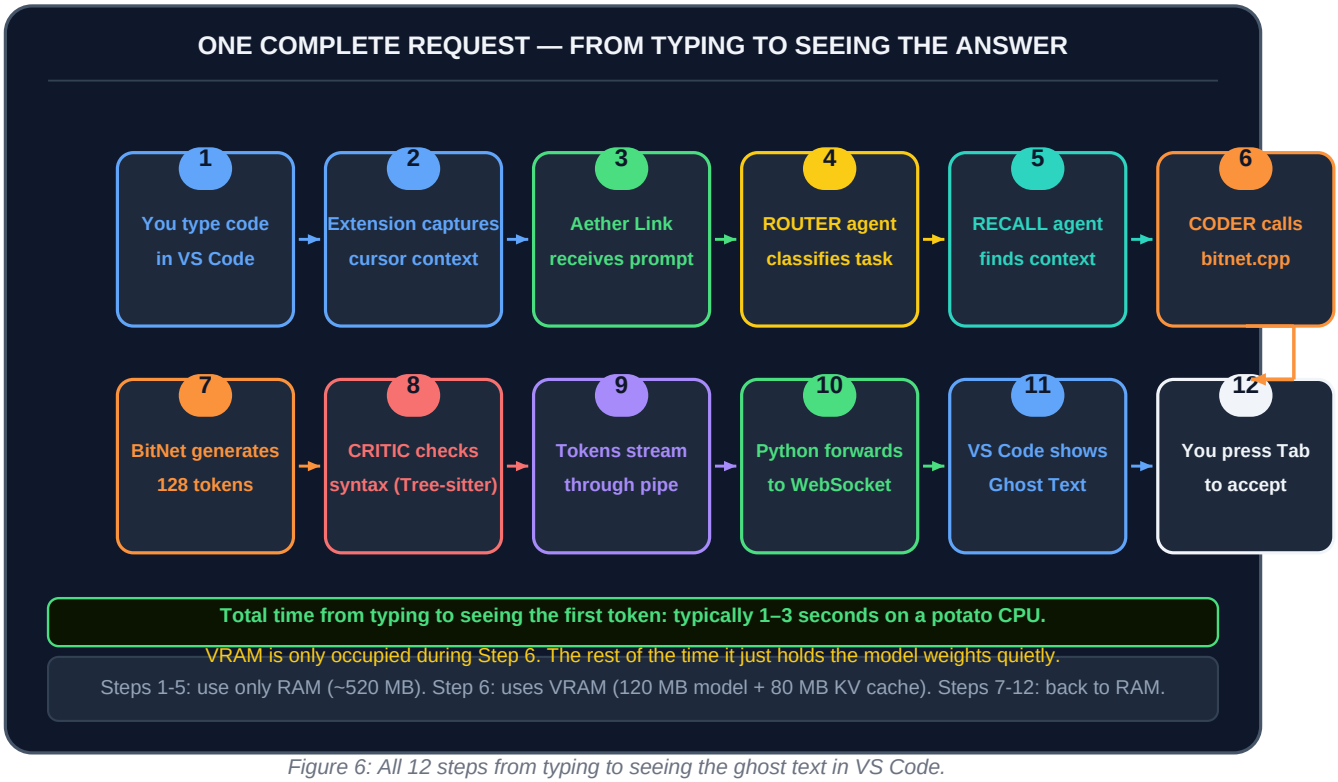

Figure 6: All 12 steps from typing to seeing the ghost text in VS Code.

8.1 Steps 1–2: The VS Code Side

When you stop typing for about 300 milliseconds, the VS Code extension triggers. It collects: your current file path, the code around the cursor (about 200 characters before and after), any recent edits, and your explicit prompt if you typed one. This context is packaged as a JSON object and sent to the Python engine. The extension then shows a loading indicator and waits.

8.2 Steps 3–5: Routing and Retrieval

Aether Link receives the JSON and passes it to the PicoClaw Orchestrator. The ROUTER agent checks the intent — is this a code generation request, a debug request, an explanation request? It determines this by looking for keyword patterns in the prompt. No AI inference needed — just a dictionary lookup. The RECALL agent then searches the Clara index (your indexed project files) for the most relevant existing code. This uses TF-IDF cosine similarity — again, no AI model involved. It returns the top 3 matching snippets.

8.3 Step 6: The One Model Call

The CODER agent assembles a prompt from: the retrieved context (top 3 snippets), the task type from the ROUTER, and the original user input. This combined prompt is kept under 300 tokens. The agent sends an

HTTP POST to bitnet.cpp on port 8088. bitnet.cpp processes the prompt through all 24 transformer layers and generates up to 128 tokens of output. This is the only step in the entire pipeline that uses the AI model.

8.4 Steps 7–9: Validation and Delivery

The CRITIC agent receives the generated code and runs two checks: Tree-sitter parses it as Python (or whichever language) and flags any syntax errors. The Shannon entropy of the output logits is checked — a sudden spike suggests the model was confused (hallucinating). If either check fails, one re-generation is triggered. If the output passes, the tokens are written to the named pipe. Python reads from the pipe and forwards the text to the WebSocket. VS Code receives the tokens and displays them as ghost text.

8.5 Step 12: Acceptance

If you press Tab, the ghost text is accepted and inserted into your editor. If you keep typing, the ghost text disappears and a new suggestion cycle begins when you pause again. The model is still sitting in memory, ready for the next request. No reloading needed.

PART 9 PicoClaw Agents Four Specialists Doing One Job Each

The PicoClaw system is the brain of the operation. It breaks the problem of 'answer the user's coding question' into four sub-problems, each handled by the cheapest possible tool. Only one of the four ever calls the AI model.

9.1 Why Micro-Agents Instead of One Big Model?

A large language model could theoretically handle everything — routing, retrieval, generation, and validation — but calling it for every subtask would be 4× slower and use 4× as many tokens. Instead, we use the principle of the right tool for the right job. Routing a request does not require intelligence — a lookup table works. Finding similar code does not require intelligence — TF-IDF works. Checking Python syntax does not require intelligence — Tree-sitter works. Only generating new code requires intelligence — and for that, we call the model once.

ROUTER (zero AI calls)	Receives the raw user intent and classifies it into one of five categories: CODE_GEN, DEBUG, EXPLAIN, REFACTOR, or SEARCH. Uses a simple dictionary of trigger keywords. 'write', 'create', 'implement' → CODE_GEN. 'bug', 'error', 'crash' → DEBUG. Classification takes under 1 millisecond. Also sets the task priority — 'urgent' or 'critical' triggers priority 1 (processes first). No neural network. No model inference. Pure Python.
RECALL (zero AI calls)	Searches the Clara index for the 3 code snippets most relevant to the user's request. Uses TF-IDF cosine similarity — converts the query and all indexed documents into 384-dimensional vectors, then finds the closest matches by computing dot products. The entire index lives on disk (sqlite-vec) and is queried via SQL. Typical latency: 2–5 ms. The retrieved snippets become context in the prompt sent to the CODER.
CODER (one AI call)	The only agent that calls bitnet.cpp. Assembles a prompt from the ROUTER's task type, the RECALL agent's context snippets, and the original user input. Keeps the total prompt under 300 tokens by truncating context if necessary. Sends an HTTP POST to localhost:8088 and waits for the generated text. Returns the text to the CRITIC. Latency: 1–3 seconds.
CRITIC (zero AI calls)	Validates the CODER's output without calling the model. Step 1: passes the code through Tree-sitter's Python parser and checks for ERROR nodes — these indicate a syntax error. Step 2: checks the Shannon entropy of the output token probabilities — a spike above 1.8× the rolling baseline suggests hallucination. If either check fails, the CRITIC sends the output back to the CODER with an augmented prompt asking it to fix the issue. Maximum one re-generation per request.

9.2 Serial Execution — One at a Time

Unlike the 16 GB version of this project which can run agents in parallel, the Potato Edition runs agents one at a time. ROUTER runs first, then RECALL, then CODER, then CRITIC. This is a deliberate choice: parallel execution would require multiple simultaneous model inference contexts, consuming more VRAM. Serial execution uses a single model context at a time. The priority queue ensures urgent tasks still jump ahead of less important ones.

PART 10 Clara Context Oracle Searching Code Without a Neural Model

Clara is the codebase memory system. It indexes your project files so that when you ask for a code suggestion, the engine can find and include relevant existing code as context. Better context = better suggestions.

10.1 What TF-IDF Is

TF-IDF stands for Term Frequency-Inverse Document Frequency. Despite the academic name, the idea is straightforward. TF (term frequency) measures how often a word appears in a document — 'fibonacci' appearing 10 times in a file is a strong signal that the file is about fibonacci. IDF (inverse document frequency) penalises words that appear in many documents — 'def' appears in every Python file, so it tells us nothing useful about a specific file. The product TF × IDF gives each word a score that reflects how distinctive it is for that particular document.

To represent a document as a vector, we take the top 384 most distinctive words and their TF-IDF scores. This gives us a 384-dimensional vector for every file in the project. Two files about similar topics will have similar vectors. Finding the most relevant files for a query means: compute the TF-IDF vector for the query, then find the files whose vectors have the smallest angle to the query vector (cosine similarity). This entire process takes about 2–5 ms.

10.2 How sqlite-vec Handles the Search

sqlite-vec is an extension to SQLite (the built-in Python database) that adds vector similarity search. We store each document's 384-dim TF-IDF vector as a binary blob in a SQLite table. When searching, we pass the query vector and sqlite-vec computes cosine distance for every document in a single SQL query. The index lives on disk — only the active query and its results touch RAM. Memory cost: about 20 MB. Compare this to ChromaDB (which this replaces) at 2 GB idle.

10.3 Why Not Neural Embeddings?

Modern retrieval systems often use neural embedding models (like sentence-transformers) to convert text into vectors that capture semantic meaning. These produce better search results but require a 400–600 MB model that adds 0.5–2 seconds per query. For code search, TF-IDF is often good enough — code uses precise technical terms (function names, library names, variable names) that TF-IDF handles very well. We trade a small amount of retrieval quality for 580 MB of RAM and 2 seconds of latency.

10.4 Tree-sitter — Parsing Code as a Tree

Tree-sitter is a fast, incremental parser for programming languages. Where a simple text search would look for the string 'def fibonacci', Tree-sitter parses the source file into an Abstract Syntax Tree (AST) — a structured tree where each node represents a language construct. Function definitions, class declarations, import statements, and variable assignments are all proper tree nodes with defined types. Clara uses Tree-sitter to extract just the function signatures from each file and include them as high-priority context. This is also how the CRITIC checks whether generated code is syntactically valid — it looks for 'ERROR' nodes in

the AST.

PART 11 The C++ Extensions What They Do (Without Writing C++)

This project includes three C++ source files. You do not need to write or understand C++ to use them. You compile them once with a single command, and after that they are imported into Python like any other module. This section explains what each one does and why Python alone cannot do it.

11.1 Why C++ at All?

Python is excellent for orchestration and logic but has limitations at the hardware level. Three specific operations cannot be done reliably from pure Python: atomic GPU-level semaphores (Python's Lock does not talk to the CUDA hardware), zero-copy byte writing to a named pipe without Python's GIL (global interpreter lock) getting in the way, and direct DMA-style memory operations. The C++ extensions are small, focused, and well-tested. Think of them as Python plugins.

vram_guard.cpp ~60 lines	Prevents two operations from accessing VRAM simultaneously. Imagine two agents both trying to write to the KV cache at the same instant — without coordination this corrupts the cache and causes incorrect outputs or crashes. vram_guard.cpp uses a C++ std::atomic_flag — a hardware-level lock that is visible to the CUDA driver. Python's threading.Lock works at the Python level but is invisible to the GPU hardware. Usage in Python: epsilon_core.vram_guard_acquire() before VRAM writes, epsilon_core.vram_guard_release() after.
token_pipe.cpp ~40 lines	Writes generated tokens to a named pipe (a tube between processes) without going through Python's garbage-collected memory. When bitnet.cpp generates a token, we want to forward it to VS Code immediately — zero buffering, zero delay. Python's file.write() adds overhead. The C++ write() system call is direct and immediate. Usage in Python: epsilon_core.pipe_send_token(token_bytes) — that is all you call.
bindings.cpp ~30 lines	The glue between C++ and Python. Uses pybind11 (a library that makes C++ callable from Python) to expose the functions from the other two files. This file defines what the Python-visible module is called (epsilon_core) and which C++ functions it exposes. You never call this file directly — the compiler produces a .so file that Python imports.

11.2 How to Compile the Extensions (One Command)

You do not need to understand the compilation process. The compile_cpp.sh script does everything:

scripts/compile_cpp.sh — one command to compile everything

```
# Run this once from the project root. Takes about 30 seconds.
bash scripts/compile_cpp.sh

# It does the following (you do not need to understand this):
#   1. cmake -B build -S .       (configure the build)
#   2. cmake --build build        (compile C++ to .so file)
#   3. cp build/epsilon_core.so backend/   (put it where Python can find it)

# Test that it worked:
python3 -c 'import epsilon_core; print("C++ extensions loaded OK")'

# After that, in Python you just do:
# import epsilon_core
# epsilon_core.vram_guard_acquire()   # acquire the VRAM lock
# ... do VRAM operations ...
# epsilon_core.vram_guard_release()   # release the lock
```

■ If you are running in CPU-only mode (no CUDA), vram_guard.cpp becomes a no-op — its functions exist but do nothing, because there is no GPU to guard. token_pipe.cpp still works — it writes to stdout instead of a named pipe. You only need CUDA installed if you explicitly want GPU acceleration.

PART 12 tinygrad and Sparse Attention Shrinking the Memory Footprint

12.1 What tinygrad Is

tinygrad is a minimalist machine learning framework created by George Hotz. It can do everything PyTorch can for our purposes — tensor operations, GPU acceleration, automatic differentiation — but with a tiny codebase and minimal dependencies. Importing PyTorch uses about 800 MB of RAM. Importing tinygrad uses about 50 MB. For our KV cache operations (which are just array indexing and matrix multiplication), tinygrad is more than sufficient and the memory saving is enormous.

12.2 The KV Cache Code in Plain Terms

Here is the entire KV cache implementation. Even if you are not familiar with machine learning, you can follow this as regular Python code:

backend/inference/tinygrad_kv.py — annotated for beginners

```python
# backend/inference/tinygrad_kv.py
# The KV cache is basically a big table that stores vectors.
# Think of it as: a spreadsheet with rows = layers, cols = tokens.
from tinygrad.tensor import Tensor

class INT8KVCache:
    def __init__(self, n_layers=24, n_heads=14, ctx=512, d_head=64):
        # Allocate two tables: one for Keys, one for Values.
        # dtype='int8' means each number stored as 1 byte (not 2 = FP16).
        # Shape: (layers, heads, max_tokens, dimensions_per_head)
        self.k   = Tensor.zeros(n_layers, n_heads, ctx, d_head, dtype='int8')
        self.v   = Tensor.zeros(n_layers, n_heads, ctx, d_head, dtype='int8')
        self.pos = 0    # Current write position (like a cursor in the table)
        mb = (self.k.nbytes() + self.v.nbytes()) / (1024 ** 2)
        print(f'KV Cache ready: {mb:.0f} MB allocated (INT8 format)')

    def write(self, layer, keys, values):
        # Store the key and value vectors for this new token.
        # INT8 means we round each float to the nearest integer (-128 to 127).
        self.k[layer, :, self.pos] = keys.cast('int8')
        self.v[layer, :, self.pos] = values.cast('int8')
        # Move the cursor forward. When we reach the end, wrap back to start.
        # This is called a 'ring buffer' — like a circular tape.
        self.pos = (self.pos + 1) % self.k.shape[2]

    def read(self, layer, n_tokens):
        # Retrieve the stored keys and values for n_tokens past tokens.
        # Cast back to float32 before doing attention math.
        return (self.k[layer, :, :n_tokens].cast('float32'),
                self.v[layer, :, :n_tokens].cast('float32'))
```

12.3 Sparse Attention in Plain Terms

The standard attention operation says: for each token, look at ALL other tokens and decide how much attention to pay to each. With 512 tokens this means 512 × 512 = 262,144 comparisons per layer. Our sparse version says: for each token, only look at the 64 tokens it finds most relevant. This means 512 × 64 = 32,768 comparisons — 8 times fewer. The 'most relevant' 64 are found by computing which ones have the highest similarity scores (dot products) with the current token's query vector.

PART 13 Python Concepts You Need A Practical Guide for This Project

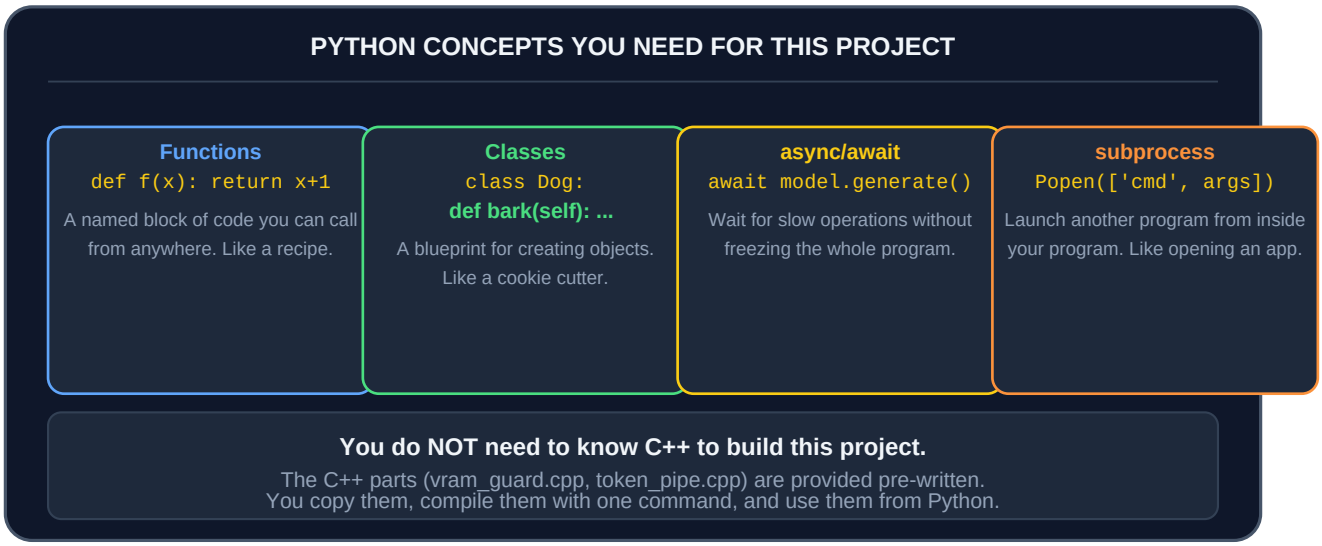

Figure 7: The four Python concepts that appear throughout this project's code.

13.1 Classes and Objects — The Building Blocks

Almost every component in this project is a class. A class is a blueprint for creating objects that bundle data and functions together. For example, BitNetModel is a class. When you create a BitNetModel object, it starts the bitnet.cpp server subprocess and stores a reference to it. When you call model.generate(), it sends an HTTP request to that subprocess. All the complexity is hidden inside the class.

Python classes — explained with BitNetModel

```python
# How classes work — simplified example
class BitNetModel:
    # __init__ is called when you create the object
    def __init__(self, model_path: str):
        self.path = model_path          # Store data on the object
        self._start_server()            # Do setup work

    def _start_server(self):
        # Methods starting with _ are 'private' — internal helpers
        import subprocess
        self.process = subprocess.Popen([...])  # Start bitnet.cpp

    def generate(self, prompt: str) -> str:
        # Public method — this is what users of the class call
        import requests
        response = requests.post('http://localhost:8088/completion',
                                 json={'prompt': prompt})
        return response.json()['content']

# Using the class:
model = BitNetModel('models/qwen-0.5b.gguf')   # Creates object, starts server
answer = model.generate('def hello():')          # Calls the model
print(answer)                                    # Prints the generated code
```

13.2 async/await — Not Freezing While Waiting

When Python calls model.generate(), it has to wait 1–3 seconds for bitnet.cpp to finish. Normally, Python would freeze completely during this wait. With async/await, Python says 'I am waiting for this — let me know when it is done' and goes back to handling other work. When generation finishes, Python resumes exactly

where it left off. For a coding assistant that might receive multiple requests quickly, this is essential.

```
# async/await — simplified explanation
import asyncio

# 'async def' means this function can pause and wait without blocking
async def handle_request(prompt: str) -> str:
    print('Starting generation...')
    # 'await' means: start this, then pause here until it is done
    # While waiting, Python can handle other requests
    result = await generate_with_model(prompt)
    print('Generation complete!')
    return result

async def generate_with_model(prompt: str) -> str:
    # asyncio.sleep simulates a slow operation (like model inference)
    await asyncio.sleep(2.0)    # Waits 2 seconds without blocking
    return 'def hello(): print("Hello, World!")'

# Run it:
result = asyncio.run(handle_request('def hello():'))
```

13.3 subprocess — Running Programs from Python

subprocess.Popen lets you start another program from inside Python and communicate with it. We use this to run bitnet.cpp. Python starts the C++ binary, then communicates with it over HTTP. If bitnet.cpp crashes, Python detects this and can restart it. The subprocess runs independently — it has its own memory, its own CPU time, and its own crash boundary.

```
import subprocess, time

# Start bitnet.cpp as a background process
# It will keep running until we stop it
process = subprocess.Popen([
    './BitNet/build/bin/llama-server',  # The program to run
    '-m', 'models/qwen-0.5b.gguf',      # First argument: model path
    '-c', '512',                         # Context size
    '--port', '8088',                    # HTTP server port
])

time.sleep(2.0)  # Wait for server to start up

# Check if it is still running
if process.poll() is None:
    print('bitnet.cpp is running!')
else:
    print('bitnet.cpp crashed!')

# Stop it when done
process.terminate()
```

PART 14 Setting Up Your Machine Complete Installation Guide

■ Before writing any code, you need to install the tools. Read through all steps first, then execute them in order. If any step fails, read the error message carefully — it almost always tells you exactly what is missing.

14.1 What You Need to Install

Tool	What It Is	Why We Need It	How to Check
Ubuntu 22.04 Server	The operating system	Server install has no desktop — saves 300–500 MB RAM immediately	cat /etc/os-release
Python 3.11	The main programming language	Everything except bitnet.cpp is Python	python3.11 --version
git	Code version control tool	To clone repositories from GitHub	git --version
cmake + build-essential	C++ build tools	To compile the C++ extensions and bitnet.cpp	cmake --version
pybind11-dev	Python-C++ bridge library	To compile the Python-callable C++ extensions	python3 -c 'import pybind11'

14.2 Step-by-Step Installation

Full installation sequence — run these commands in order

```
# STEP 1 — Update the system and install build tools
# This gives you git, cmake, and the C++ compiler
sudo apt update && sudo apt upgrade -y
sudo apt install -y git cmake build-essential python3.11 python3.11-venv python3-pip pybind11-dev

# STEP 2 — Set up a Python virtual environment
# A virtual environment keeps your project's packages separate from the system
cd ~
python3.11 -m venv epsilon-env
source epsilon-env/bin/activate   # Activate it (do this every time you work on the project)

# STEP 3 — Install Python packages
# Notice: no 'torch' or 'transformers' — those are too heavy
pip install tinygrad sqlite-vec tree-sitter tree-sitter-python
pip install huggingface-hub psutil pyyaml requests

# STEP 4 — Clone and build bitnet.cpp
git clone --recursive https://github.com/microsoft/BitNet.git
cd BitNet
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release --parallel $(nproc)
cd ..

# STEP 5 — Download the model (this downloads ~120 MB from HuggingFace)
python3 download_model.py

# STEP 6 — Set up swap space
bash scripts/setup_swap.sh

# STEP 7 — Verify everything works
python3 -c 'import tinygrad; import sqlite_vec; print("Python packages OK")'
./BitNet/build/bin/llama-server --version
```

PART 15 Building Component 1 The BitNet Model Wrapper

This is the first file you will write. It is the bridge between your Python code and the bitnet.cpp server. Once this works, you can generate code from Python with a single function call. Everything else builds on top of this.

15.1 Create the File

Create a file at **backend/tiers/bitnet_model.py**. You can use any text editor — VS Code, nano, gedit. Type or paste the following code exactly:

backend/tiers/bitnet_model.py — complete file

```python
# backend/tiers/bitnet_model.py
# This file manages the bitnet.cpp inference server.
# It starts the server, sends prompts to it, and returns the results.

import subprocess   # For starting bitnet.cpp as a background process
import time         # For waiting
import requests     # For HTTP calls to bitnet.cpp
from threading import Lock  # For making sure only one request runs at a time

class BitNetModel:
    """
    Wrapper around the bitnet.cpp inference server.
    Usage:
        model = BitNetModel('models/qwen-0.5b.gguf')
        result = model.generate('def hello():')
    """
    def __init__(self, model_path: str, context: int = 512, threads: int = 2):
        # Lock ensures only one agent calls the model at a time
        self._lock    = Lock()
        self._process = None
        self._start_server(model_path, context, threads)

    def _start_server(self, model_path: str, ctx: int, threads: int) -> None:
        """Start the bitnet.cpp HTTP server as a background process."""
        command = [
            './BitNet/build/bin/llama-server',
            '-m',          model_path,   # Path to the GGUF model file
            '-c',          str(ctx),     # Context length (512 tokens)
            '-t',          str(threads), # CPU threads to use
            '--port',      '8088',       # HTTP port
            '-ngl',        '0',          # GPU layers = 0 (use CPU only)
            '--log-disable',             # Less noisy output
        ]
        self._process = subprocess.Popen(
            command,
            stdout=subprocess.DEVNULL,  # Hide bitnet.cpp output
            stderr=subprocess.DEVNULL,
        )
        # Wait for the server to start accepting requests
        time.sleep(2.0)
        # Verify it started
        if self._process.poll() is not None:
            raise RuntimeError('bitnet.cpp server failed to start!')
        print('[BitNetModel] Server started on port 8088 (CPU mode)')

    def generate(self, prompt: str, max_tokens: int = 128,
                 temperature: float = 0.2) -> str:
        """
        Generate text given a prompt.
        max_tokens: how many tokens to generate (128 ≈ 100 words)
        temperature: 0.0 = deterministic, 1.0 = creative/random
        """
        # The Lock means: if one call is running, the next one waits
        with self._lock:
            try:
                response = requests.post(
                    'http://localhost:8088/completion',
                    json={
                        'prompt':      prompt,
                        'n_predict':   max_tokens,
                        'temperature': temperature,
                        'stop':        ['\n\n', '```'],  # Stop tokens
                    },
                    timeout=60,  # If no response in 60s, give up
                )
                return response.json()['content']
            except requests.Timeout:
                return '# Error: model took too long to respond'
            except Exception as e:
                return f'# Error: {e}'

    def shutdown(self) -> None:
        """Stop the bitnet.cpp server cleanly."""
        if self._process:
            self._process.terminate()
            self._process.wait()
            print('[BitNetModel] Server stopped')
```

15.2 Test It Works

test_model.py — verify the model wrapper works

```python
# Save this as test_model.py and run: python3 test_model.py
from backend.tiers.bitnet_model import BitNetModel

print('Loading model...')
model = BitNetModel('models/qwen-0.5b/ggml-model-i2_s.gguf')

print('Generating...')
result = model.generate('def fibonacci(n: int) -> int:')

print('Result:')
print(result)

model.shutdown()

# Expected output:
# [BitNetModel] Server started on port 8088 (CPU mode)
# Result:
#     if n <= 1:
#         return n
#     return fibonacci(n-1) + fibonacci(n-2)
```

PART 16 Building Component 2 The PicoClaw Orchestrator

With the model wrapper working, we can now build the agent system on top of it. The orchestrator receives a request and routes it through the four agents in sequence.

backend/picoclaw/potato_orchestrator.py — Part 1: routing + dispatch

```
# backend/picoclaw/potato_orchestrator.py
# The orchestrator receives requests and dispatches them through agents.
import asyncio, gc
from tree_sitter import Language, Parser

# The ROUTER's keyword dictionary
# Keys are task types. Values are lists of trigger words.
ROUTES = {
    'CODE_GEN':  ['write','create','implement','generate','function','class','def'],
    'DEBUG':     ['bug','error','crash','exception','traceback','fix','broken'],
    'EXPLAIN':   ['explain','what','how','why','describe','tell me'],
    'REFACTOR':  ['refactor','clean','improve','optimise','simplify'],
    'SEARCH':    ['find','search','where','locate','show me'],
}

class PotatoOrchestrator:
    def __init__(self, model, clara):
        self.model  = model   # BitNetModel instance
        self.clara  = clara   # PotatoClaraOracle instance
        self._setup_treesitter()

    def _setup_treesitter(self):
        """Set up the Python parser for the CRITIC agent."""
        from tree_sitter import Language, Parser
        Language.build_library('ts_langs.so', ['vendor/tree-sitter-python'])
        py_lang     = Language('ts_langs.so', 'python')
        self.parser = Parser()
        self.parser.set_language(py_lang)

    # ■■ ROUTER AGENT ■■■
```

backend/picoclaw/potato_orchestrator.py — full implementation

```
# (potato_orchestrator.py continued)


        # Step 2: RECALL — zero model calls
        # Search the project for relevant existing code
        context_docs = self.clara.search(intent, k=3)
        context_text = '\n'.join(doc['preview'] for doc in context_docs)

        # Step 3: Build a tight prompt (under 300 tokens)
        # The model only sees: context + task + user input
        prompt = (
            f'### Relevant code in project:\n{context_text[:400]}\n\n'
            f'### Task type: {task_type}\n'
            f'### Request: {intent[:200]}\n\n'
            f'### Code:'
        )

        # Step 4: CODER — exactly one model call
        result = self.model.generate(prompt, max_tokens=128, temperature=0.2)

        # Step 5: CRITIC — zero model calls
        # If output is code and has syntax errors, try once more
        if task_type == 'CODE_GEN' and not self.is_valid_syntax(result):
            fix_prompt = prompt + result + '\n# Fix the syntax error above:\n'
            result = self.model.generate(fix_prompt, max_tokens=64)

        gc.collect()  # Clean up memory after each request
        return result
```

PART 17 Building Component 3 Clara Context Oracle

Clara indexes your project's code files and makes them searchable. Every time you make a code suggestion request, Clara finds the most relevant existing code to include as context. Better context means the model generates more appropriate suggestions.

backend/clara/potato_oracle.py — Part 1: setup + TF-IDF

```python
# backend/clara/potato_oracle.py
# Clara indexes code files using TF-IDF and stores them in sqlite-vec.
# TF-IDF is a classical search algorithm — no neural network needed.
import sqlite3, sqlite_vec, struct, math, hashlib
from collections import Counter
from pathlib import Path

class PotatoClaraOracle:
    DIM = 384  # Size of each document's vector (384 numbers)

    def __init__(self, db_path: str = 'clara.db'):
        # Connect to (or create) the SQLite database
        self.conn = sqlite3.connect(db_path)
        # Load the sqlite-vec extension that adds vector search
        self.conn.enable_load_extension(True)
        sqlite_vec.load(self.conn)
        self._create_table()
        self.vocab: dict = {}   # word → index mapping
        self.idf:   dict = {}   # word → IDF score mapping

    def _create_table(self) -> None:
        """Create the database table if it does not exist."""
        self.conn.execute('''
            CREATE TABLE IF NOT EXISTS docs (
                path      TEXT PRIMARY KEY,
                content   TEXT NOT NULL,
                embedding BLOB NOT NULL
            )
        ''')
        self.conn.commit()

    def _tfidf(self, text: str) -> list:
        """
        Convert text to a 384-dimensional TF-IDF vector.
        Each position in the vector represents one word.
        The value = how important that word is for this text.
        """
        # Count word occurrences
        words  = text.lower().split()
        counts = Counter(words)
        # Build the vector: 384 zeros, then fill in TF-IDF scores
        vector = [0.0] * self.DIM
        for word, count in counts.items():
            if word in self.vocab and self.vocab[word] < self.DIM:
                idx        = self.vocab[word]
                tf_score   = count / max(len(words), 1)     # How often in THIS doc
                idf_score  = self.idf.get(word, 1.0)        # How rare across ALL docs
                vector[idx] = tf_score * idf_score
        # Normalise so all vectors have length 1 (required for cosine similarity)
        magnitude = math.sqrt(sum(x * x for x in vector)) or 1.0
        return [x / magnitude for x in vector]

    def index_file(self, path: str, content: str) -> None:
        """Add a file to the search index."""
        vector = self._tfidf(content)
        # Pack the 384 floats into bytes for database storage
        blob = struct.pack(f'{self.DIM}f', *vector)
        self.conn.execute(
            'INSERT OR REPLACE INTO docs (path, content, embedding) VALUES (?,?,?)',
            (path, content[:1500], blob)   # Only store first 1500 chars
        )
        self.conn.commit()

    def search(self, query: str, k: int = 3) -> list:
        """
        Find the k most relevant documents for a query.
    # ... (continued)
```

backend/clara/potato_oracle.py — full implementation

```
# (potato_oracle.py continued)

        Uses cosine similarity: smaller distance = more similar.
        """
        query_vec  = self._tfidf(query)
        query_blob = struct.pack(f'{self.DIM}f', *query_vec)
        # sqlite-vec does the cosine distance calculation in SQL
        rows = self.conn.execute('''
            SELECT path, content, vec_distance_cosine(embedding, ?) AS dist
            FROM docs
            ORDER BY dist ASC
            LIMIT ?
        ''', (query_blob, k)).fetchall()
        return [{'path': r[0], 'preview': r[1][:200], 'score': 1.0 - r[2]}
                for r in rows]

    def crawl(self, directory: str) -> int:
        """Index all code files in a directory."""
        count = 0
        for ext in ('.py', '.js', '.ts', '.md'):
            for file_path in Path(directory).rglob(f'*{ext}'):
                if file_path.stat().st_size > 100_000:   # Skip files > 100KB
                    continue
                try:
                    text = file_path.read_text(errors='ignore')
                    self.index_file(str(file_path), text)
                    count += 1
                except Exception:
                    pass
        print(f'[Clara] Indexed {count} files')
        return count
```

PART 18 Building Component 4 Aether Link — The Communication Bridge

Aether Link is the async event loop that sits between VS Code and the PicoClaw orchestrator. It reads requests from stdin, dispatches them, and writes responses to stdout. It is intentionally simple — no web framework, no HTTP server.

backend/aether/aether_link.py — the communication bridge

```python
# backend/aether/aether_link.py
# Reads JSON from stdin, dispatches through PicoClaw, writes JSON to stdout.
import asyncio, json, sys

class AetherLink:
    """
    Minimal communication bridge.
    Input:  one JSON object per line on stdin
    Output: one JSON object per line on stdout
    """
    def __init__(self, orchestrator):
        self.orchestrator = orchestrator

    async def run(self) -> None:
        """Main loop — read, process, respond, repeat."""
        # Set up async reading from stdin
        reader = asyncio.StreamReader()
        protocol = asyncio.StreamReaderProtocol(reader)
        await asyncio.get_event_loop().connect_read_pipe(
            lambda: protocol, sys.stdin
        )
        print('[AetherLink] Ready — listening on stdin', file=sys.stderr)

        # Process one request at a time
        async for raw_bytes in reader:
            line = raw_bytes.decode().strip()
            if not line:
                continue

            # Parse the incoming JSON
            try:
                request = json.loads(line)
            except json.JSONDecodeError:
                self._respond({'ok': False, 'error': 'Invalid JSON'})
                continue

            # Dispatch to the orchestrator
            try:
                prompt = request.get('prompt', '')
                result = await self.orchestrator.dispatch(prompt)
                self._respond({'ok': True, 'result': result})
            except Exception as e:
                self._respond({'ok': False, 'error': str(e)})

    def _respond(self, data: dict) -> None:
        """Write a JSON response to stdout, followed by a newline."""
        # flush=True ensures VS Code receives the response immediately
        print(json.dumps(data), flush=True)
```

PART 19 Building Component 5 The Main Engine — Wiring It All Together

The main engine creates all the components, connects them, and runs the event loop. This is the file you will run to start the entire system.

backend/main.py — the complete entry point

```python
# backend/main.py — Start the entire Epsilon Potato Engine
# Run with: python3 backend/main.py
import asyncio, sys, psutil, gc, yaml
from tiers.bitnet_model import BitNetModel
from picoclaw.potato_orchestrator import PotatoOrchestrator
from clara.potato_oracle import PotatoClaraOracle
from inference.tinygrad_kv import INT8KVCache
from aether.aether_link import AetherLink

def print_memory(label: str) -> None:
    """Print current RAM usage so we can track memory consumption."""
    ram = psutil.Process().memory_info().rss / (1024 ** 2)
    print(f'  [{label}] RAM used: {ram:.0f} MB / 2048 MB', file=sys.stderr)

def main():
    # Load configuration from config.yaml
    with open('config.yaml') as f:
        config = yaml.safe_load(f)

    print('=== EPSILON POTATO ENGINE STARTING ===', file=sys.stderr)
    print_memory('start')

    # Create all components in order
    # Each print_memory() call shows the RAM cost of each component

    print('[1/4] Loading BitNet model...', file=sys.stderr)
    model = BitNetModel(
        model_path = config['model_path'],
        context    = config.get('context_len', 512),
        threads    = config.get('cpu_threads', 2),
    )
    print_memory('model loaded')

    print('[2/4] Building Clara index...', file=sys.stderr)
    clara = PotatoClaraOracle(config.get('db_path', 'clara.db'))
    if config.get('crawl_on_start') and config.get('project_dir'):
        clara.crawl(config['project_dir'])
    print_memory('clara ready')

    print('[3/4] Allocating KV cache...', file=sys.stderr)
    kv = INT8KVCache()   # 80 MB allocated here
    print_memory('kv cache ready')

    print('[4/4] Starting PicoClaw orchestrator...', file=sys.stderr)
    orchestrator = PotatoOrchestrator(model, clara)
    print_memory('orchestrator ready')

    print('=== ENGINE READY — Waiting for requests ===', file=sys.stderr)

    # Start the event loop
    link = AetherLink(orchestrator)
    try:
        asyncio.run(link.run())
    except KeyboardInterrupt:
        print('\nShutting down...', file=sys.stderr)
    finally:
        model.shutdown()
        print('Engine stopped.', file=sys.stderr)

if __name__ == '__main__':
    main()
```

19.1 The config.yaml File

config.yaml — all settings in one place

```
# config.yaml — put this in the project root
model_path:    models/qwen-0.5b/ggml-model-i2_s.gguf
context_len:   512       # Tokens per request
cpu_threads:   2         # CPU threads for inference
db_path:       clara.db  # Clara search index location
project_dir:   /home/user/my_project    # Your code to index
crawl_on_start: true     # Index project files when engine starts
```

19.2 Running the Engine

Running and testing the engine

```
# Start the engine (runs until you press Ctrl+C)
python3 backend/main.py

# In a different terminal, send a test request:
echo '{"prompt": "def bubble_sort(arr):"}' | python3 backend/main.py

# Expected response:
# {"ok": true, "result": "    n = len(arr)\n    for i in range(n):\n..."}
```

PART 20 Testing, Debugging, and Monitoring Knowing When Things Go Wrong

20.1 The Most Common Errors and What They Mean

Error Message	What It Means	How to Fix It
ModuleNotFoundError: No module named 'tinygrad'	Python cannot find the tinygrad package	Make sure you activated the virtual environment: source epsilon-env/bin/activate
ConnectionRefusedError: localhost:8088	bitnet.cpp server is not running	Check that llama-server started. Try: ./BitNet/build/bin/llama-server --version
RuntimeError: bitnet.cpp server failed to start	The bitnet.cpp binary crashed on launch	Check the model path in config.yaml is correct and the file exists
MemoryError: Insufficient VRAM	GPU is out of memory	Set n_gpu_layers to 0 in config (use CPU mode). PyTorch may have been imported.
ERROR nodes in Tree-sitter parse	Generated code has syntax errors	Normal — CRITIC will trigger re-generation. If it happens every time, lower temperature.
Swap usage > 1 GB	System is running low on RAM	Check what is using memory: psutil.Process().memory_info(). Something loaded PyTorch.

20.2 The Monitoring Script

scripts/monitor.py — real-time health monitoring

```python
# scripts/monitor.py — run while the engine is active to watch memory
import psutil, time, subprocess

def check_health():
    # RAM usage of this process
    ram_mb = psutil.Process().memory_info().rss / (1024 ** 2)
    print(f'RAM:  {ram_mb:.0f} MB (limit: 2048 MB)')
    if ram_mb > 1800:
        print('  WARNING: Dangerously close to RAM limit!')

    # Swap usage
    swap = psutil.swap_memory()
    swap_mb = swap.used / (1024 ** 2)
    print(f'SWAP: {swap_mb:.0f} MB (should be near 0 normally)')
    if swap_mb > 500:
        print('  WARNING: Using significant swap — system is under memory pressure')

    # GPU VRAM (only works with NVIDIA GPU and CUDA)
    try:
        out = subprocess.check_output([
            'nvidia-smi',
            '--query-gpu=memory.used,memory.free',
            '--format=csv,noheader,nounits'
        ]).decode().strip().split(', ')
        vram_used, vram_free = out
        print(f'VRAM: {vram_used} MB used, {vram_free} MB free')
        if int(vram_used) > 1800:
            print('  WARNING: VRAM near limit — risk of CUDA OOM crash!')
    except FileNotFoundError:
        print('VRAM: No NVIDIA GPU detected (CPU mode)')

# Run checks every 5 seconds
while True:
    print('■' * 40)
    check_health()
    time.sleep(5)
```

20.3 How to Benchmark Performance

scripts/benchmark.py — measure inference speed

```python
# scripts/benchmark.py — measure tokens per second
import time
from backend.tiers.bitnet_model import BitNetModel

model = BitNetModel('models/qwen-0.5b/ggml-model-i2_s.gguf')

# Warm-up run (first inference is slower due to cache effects)
model.generate('def hello():', max_tokens=20)

# Timed run
prompt = 'def quicksort(arr: list) -> list:'
start  = time.perf_counter()
result = model.generate(prompt, max_tokens=80)
elapsed = time.perf_counter() - start

tokens = len(result.split())  # Approximate token count
print(f'Generated {tokens} tokens in {elapsed:.2f} seconds')
print(f'Speed: {tokens / elapsed:.1f} tokens/second')
print(f'Expected for potato CPU: 30–45 tokens/second')
print(f'Result preview: {result[:100]}')

model.shutdown()
```

GLOSSARY — Every Technical Term Explained in Plain English

Every term in this guide is defined here in plain language. If you encounter an unfamiliar word while reading, look it up here first.

Abstract Syntax Tree (AST)	A tree-shaped representation of code's structure. 'def fibonacci(n)' becomes a node of type 'function_definition' with children for the name 'fibonacci' and the parameter 'n'. Tree-sitter generates ASTs instantly and we use them to check whether generated code is syntactically valid.
Activation buffer	Temporary memory used during model inference. When a token flows through a transformer layer, the intermediate values (activations) are stored briefly, then discarded after that layer is done. We pre-allocate a 50 MB activation buffer to avoid repeated malloc/free calls.
async/await	Python keywords that allow a function to pause and wait for a slow operation without freezing the entire program. 'async def' declares an asynchronous function. 'await' pauses until the awaited operation finishes.
Attention head	One copy of the attention mechanism in a transformer layer. Qwen 0.5B has 14 heads per layer. Each head learns to look at different aspects of the input simultaneously.
bitnet.cpp	Microsoft's open-source C++ inference engine specifically designed for BitNet ternary-quantised models. It compiles to a standalone server binary that accepts HTTP requests. Far lighter than PyTorch.
BitNet 1.58-bit	A quantisation scheme where every model weight is stored as one of three values: –1, 0, or +1. Named for log■(3) = 1.58 bits of information per weight. Models must be trained from scratch with this constraint.
CUDA	NVIDIA's programming framework for GPU computation. Required for GPU acceleration. In our project, CUDA is optional — we run in CPU mode by default.
CUDA OOM	CUDA Out Of Memory error. Occurs when your GPU VRAM is full and a new allocation is requested. Can crash the display driver. Our 390 MB VRAM usage leaves 1.7 GB of safety margin on a 2 GB card.
Context length	The maximum number of tokens the model considers at once. We use 512. Longer contexts require more KV cache memory. 512 tokens ≈ 380 words.
Cosine similarity	A way to measure how similar two vectors are, based on the angle between them. Two identical vectors have cosine similarity = 1. Two completely different vectors have cosine similarity near 0. Used in TF-IDF and sqlite-vec search.
Event loop	Python's asyncio mechanism for managing multiple concurrent tasks without threads. The event loop keeps a queue of callbacks and executes them one at a time, switching between tasks at await points.
FP16 (float16)	16-bit floating-point number format. Standard for storing neural network weights. Each weight uses 2 bytes. 494M weights × 2 bytes = 988 MB. BitNet replaces this with ternary values stored in 0.25 bytes per weight.

GGUF	A file format for storing quantised language models. Stands for GPT-Generated Unified Format. Used by llama.cpp and bitnet.cpp. Our model is stored as ggml-model-i2_s.gguf — the i2_s suffix means 2-bit signed integer kernel.
Ghost Text	The greyed-out code suggestion that appears in VS Code as you type, waiting for you to press Tab to accept it. Copilot shows ghost text. Our system shows ghost text too, but running locally.
GIL (Global Interpreter Lock)	Python's internal lock that allows only one thread to execute Python bytecode at a time. This is why pure Python cannot do true parallelism. C extensions (like our vram_guard.cpp) can release the GIL and run in parallel.
HTTP	HyperText Transfer Protocol — the same protocol web browsers use. We use it for Python-to-bitnet.cpp communication because it is simple and bitnet.cpp already includes an HTTP server.
Inference	Using a trained model to generate output. The opposite of training. During inference, model weights are read-only. No learning happens.
INT8	8-bit integer format. Stores values from –128 to +127. Used for our KV cache instead of FP16 to halve memory usage. Some precision is lost but accuracy stays within acceptable bounds.
KV cache	Key-Value cache. Stores the Key and Value vectors from all previous tokens so they do not need to be recomputed on every generation step. Without it, inference would be $O(n^2)$ — extremely slow for longer contexts.
Named pipe	A file-like communication channel between two processes on the same machine. Process A writes bytes to the pipe. Process B reads them immediately. Zero-copy, sub-millisecond latency. Used for token streaming.
OOM (Out of Memory)	Error that occurs when a program requests more memory than is available. RAM OOM causes the process to crash. VRAM OOM can crash the GPU driver.
Parameter	Another word for a model weight. Qwen 0.5B has 494 million parameters.
PCIe	PCI Express — the physical slot your GPU plugs into. Data moves between system RAM and VRAM through this bus at about 16 GB/s. VRAM-internal bandwidth is 500+ GB/s — 30× faster.
pybind11	A C++ library that makes C++ functions callable from Python. Used to compile vram_guard.cpp and token_pipe.cpp into a Python module.
Quantisation	Reducing the precision of model weights to save memory. FP16 → INT8 is 2-bit quantisation (saves 50%). BitNet is 1.58-bit quantisation (saves 87.5%).
SIMD	Single Instruction Multiple Data — CPU instructions that operate on multiple values simultaneously. AVX-512 (Intel/AMD) can add 64 integers in one instruction. bitnet.cpp uses SIMD to process 64 ternary weights per CPU cycle.

sparse attention	Attention that attends to only the K most relevant tokens instead of all N tokens. Reduces KV cache memory from $O(n^2)$ to $O(n \cdot k)$. We use K=64 out of 512 tokens.
SQLite	A lightweight embedded database built into Python's standard library. Stores data in a single file. We use it for the Clara index via sqlite-vec.
sqlite-vec	An extension to SQLite that adds vector similarity search. Allows storing and searching 384-dimensional vectors using SQL syntax.
stdin / stdout	Standard input and output — the default keyboard and screen of a program. We use these for VS Code ↔ Python communication instead of a web server.
subprocess	Running one program as a child of another. Python's subprocess.Popen starts bitnet.cpp as a background process and allows Python to communicate with it.
Swap space	Disk space used as overflow RAM. When RAM fills up, the OS moves cold pages to the swap file. 1000× slower than RAM but prevents immediate crashes.
Tensor	A multi-dimensional array of numbers. A vector is a 1D tensor. A matrix is a 2D tensor. Model weights and activations are stored as tensors.
TF-IDF	Term Frequency-Inverse Document Frequency. A classical algorithm for converting documents to vectors for similarity search. No neural network required. TF = how often a word appears in the document. IDF = how rare it is overall.
Token	A piece of text that the model processes as a unit. Roughly 0.75 words. Each token has a numeric ID in a vocabulary of 151,936 tokens.
Tokenisation	Converting text to token IDs before feeding to the model. Detokenisation is the reverse — converting token IDs back to text.
Transformer	The neural network architecture used by all modern language models. Consists of stacked layers, each containing self-attention and feed-forward networks.
tinygrad	A minimalist machine learning framework by George Hotz. Used instead of PyTorch for our KV cache operations. 50 MB idle RAM vs 800 MB for PyTorch.
Tree-sitter	A fast incremental code parser that generates ASTs for many programming languages. Used by the CRITIC agent to check generated code for syntax errors.
Virtual environment	An isolated Python installation for a specific project. Prevents packages from conflicting with each other. Created with: python3 -m venv myenv
VRAM (Video RAM)	Memory on your GPU chip. The GPU can access this 30× faster than system RAM. Model weights and the KV cache live here. We have 2 GB.
Weight	One number in a neural network. The model has 494 million of them. They are learned during training and fixed during inference.

WebSocket	A persistent two-way connection between a web page (or VS Code extension) and a server. Unlike HTTP, the connection stays open and either side can send messages at any time.

You now understand every part of this project — from what RAM is, to how the model fits in VRAM, to how the agents talk to each other, to how to write every piece of code. The only thing left is to build it.

Epsilon IDE Engine · Potato PC Edition · SEAL Project · The Complete Beginner's Guide